HDOC DAY – 50

Algorithm: Majority Element

1. Start.
2. Read the size of the array size.
3. Check whether size is positive.
4. Declare a variable-length integer array nums[size].
5. Read all the elements of nums[].
6. Pass the address of the array to the function majority_element().
7. In majority_element():
    - Repeat for each element nums[i].
    - Set count = 0.
    - Compare nums[i] with every element of the array.
    - If the elements are equal, increment count.
    - After counting, check whether count > size / 2.
    - If true, display nums[i] as the majority element and return from the function.
8. If no element occurs more than size / 2 times, display -1.
9. Stop.

C PROGRAM:

```
#include <stdio.h>


void majority_element(int *nums, int size)
{
   int i, j, count;
```

```
    for (i = 0; i < size; i++)
    {
        count = 0;

        // Count occurrences of nums[i]
        for (j = 0; j < size; j++)
        {
            if (*(nums + i) == *(nums + j))
            {
                count++;
            }
        }

        // Check if occurrence is strictly greater than size/2
        if (count > size / 2)
        {
            printf("Majority element: %d\n", *(nums + i));
            return;
        }
    }

    printf("-1\n");
}

int main()
{
    int size, i;
```

```
    printf("Enter the size of the array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid size.\n");
        return 0;
    }

    int nums[size];

    printf("Enter the array elements:\n");

    for (i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    // Passing the address of the array
    majority_element(nums, size);

    return 0;
}
```

Test Case Table

Test Case	Size	Array	size / 2	Expected Output
1	5	3 3 2 3 3	2	Majority element: 3
2	7	2 2 1 1 1 2 2	3	Majority element: 2

3	6	1 2 3 2 3 3	3	-1
4	1	5	0	Majority element: 5
5	4	1 1 2 2	2	-1